



University of Moratuwa

Department of Electronic & Telecommunication Engineering

EN2150 - Communication Network Engineering

Design of Routing Protocol

Adaptive Open Shortest Path First (AOSPF)

Team Synex

230033J	Amarasingha A.A.D.K.
230082G	Bandara W.D.A.C.
230164K	Dissanayake R.K.T.
230629R	Tennakoon U.G.R.B.

16th May 2026

Contents

1	Introduction	3
2	Baseline Protocols and Limitations	3
2.1	Routing Information Protocol (RIP)	3
2.2	Open Shortest Path First (OSPF)	4
2.3	Intermediate System to Intermediate System (IS-IS)	5
2.4	Border Gateway Protocol (BGP)	6
3	Proposed Protocol Design - Adaptive Open Shortest Path First (AOSPF)	7
3.1	Adaptive Composite Routing Metric	7
3.2	Threshold-Based Triggering Mechanism	7
3.3	Enhanced Control Packet Formats	8
3.3.1	Common Packet Header (Unchanged from OSPF)	8
3.3.2	Enhanced Link-Metric TLV (AOSPF Extension)	8
3.3.3	THRESHOLD_CROSSED Flag	8
3.4	Modified Simulation Components	9
3.4.1	Original OSPF Cost Calculation (in <code>_load_topology</code>)	9
3.4.2	AOSPF Modified Cost Calculation	9
3.4.3	Original <code>get_current_cost</code> Function	9
3.4.4	AOSPF Enhanced <code>get_current_cost</code> with Delay Awareness	9
3.4.5	New Threshold Monitoring in Simulation Loop	10
3.5	LS-UPDATE Flooding on Threshold Crossings	10
4	Route Computation with Adaptive Metrics	10
4.1	Modified SPF Computation	11
5	Failure Detection and Recovery	11
6	Security Enhancements	11
6.1	Overview of HMAC	11
6.2	Canonical Representation for AOSPF Packets	12
6.3	HMAC Signing and Verification Procedures	12
6.4	How HMAC Improves Protocol Robustness	13
6.5	Security Impact on Adaptive Metrics	14
7	Simulation Implementation	14
7.1	Modified Data Structures	14
7.2	Modified Event Types	14
7.3	Modified Methods	15
7.4	New Methods Added	15
8	Experiment Matrix and Parameters	15
8.1	Key Parameters	15
8.2	Performance Metrics	15
9	Results	15
9.1	Comparison: AOSPF vs OSPF	16
9.2	Specific Expectations	16

10 Limitations and Extensions	16
10.1 Limitations	16
10.2 Extensions	16
11 Router State Machine (Modified for AOSPF)	17
12 Conclusion	17

1. Introduction

This report presents a novel routing protocol that addresses the limitations of existing routing protocols by combining the strengths of them. The protocol merges OSPF's rapid link-state flooding with BGP's policy flexibility, while embedding security directly into the design. It introduces a multi-metric cost function that considers both latency and bandwidth, overcoming the single-metric constraints of traditional protocols. All control messages are authenticated and protected against replay attacks.

The project focuses on a simulated static wired network, with an optional mobile wireless extension. Implementation uses Python with NetworkX and assumes IPv4-like addressing. Deliverables include a design report, a functional simulator, performance experiment code, and a final written report with security analysis. Design decisions are guided by RFCs and academic literature, with validation against RIP, OSPF, IS-IS, and BGP baselines.

2. Baseline Protocols and Limitations

We compare key legacy protocols to identify specific limitations to address.

2.1 Routing Information Protocol (RIP)

RIP is one of the oldest dynamic routing protocols still in use, primarily designed for small, homogeneous networks. Often described as the “simplest distance-vector protocol” or the “beginner’s routing protocol,” RIP allows routers to share reachability information based solely on hop count. Its main purpose is to provide a simple, loop-free path to destination networks within a limited network diameter.

Type:

RIP is a distance-vector protocol, which means each router maintains a table of destinations along with the distance (hop count) and direction (next-hop interface) to reach them. It operates over UDP port 520, sending periodic updates without establishing a persistent connection.

Scope:

RIP is strictly an IGP, designed for routing within a single Autonomous System (AS) or a small, standalone network. It is not used between major internet service providers or across the global internet backbone. RIPv1 is classful, broadcast-based and RIPv2 is classless, multicast-based with optional authentication. RIPv6 extends support to IPv6.

Metric:

RIP uses a single, simple metric: the number of routers (hops) between the source and destination network.

Best Path = Lowest Hop Count Maximum Hops = 15 (Any route requiring 16 or more hops is considered unreachable. This hard limit caps the maximum network diameter at 15 routers.)

Key Features:

- Extreme Simplicity: Configuration typically requires only enabling RIP on interfaces and advertising networks.

- Low Resource Footprint: Requires minimal CPU and memory.
- Widely Supported
- Built-in Loop Prevention: Uses split horizon, route poisoning, and hold-down timers
- VLSM & CIDR Support (RIPv2+)

Limitations:

- Poor Scalability: The 15-hop limit makes RIP unusable for all but the smallest networks
- Slow Convergence: Periodic full-table updates every 30 seconds combined with hold-down timers (180 seconds) mean network failures can take minutes to resolve, causing temporary black-holes or loops.
- Inefficient Metric: Hop count ignores critical factors like link speed, congestion, packet loss, and administrative cost.
- Chatty Behavior: Broadcasting or multicasting the entire routing table every 30 seconds wastes bandwidth on modern high-speed networks.
- No Authentication in RIPv1
- No Support for Hierarchical Design

2.2 Open Shortest Path First (OSPF)

Type:

OSPF is a Link-State routing protocol. It is an Interior Gateway Protocol (IGP), meaning it is designed to move data within a single internal network or Autonomous System (AS). It uses the Shortest Path First (SPF) or Dijkstra's algorithm to calculate the loop-free shortest path to every node in the network.

Scope:

OSPF uses a two-layer hierarchy consisting of Areas. The core of the OSPF network is Area 0 (Backbone). All other areas must physically or logically connect to Area 0 to exchange routing information. By dividing the network into areas, OSPF limits the size of the Link-State Database (LSDB) on individual routers, reducing the processing load and containing the impact of network "flapping" (constant status changes) to a specific area.

Metric:

OSPF uses Cost as its metric. Cost is an arbitrary value inversely proportional to the bandwidth of an interface. The faster the link, the lower the cost.

$$\text{Cost} = \text{Reference Bandwidth} / \text{Interface Bandwidth}$$

Default Reference: Typically 100 Mbps. This means a 10 Mbps link has a cost of 10, while a 100 Mbps link has a cost of 1.

Key Features:

- Fast Convergence: Because routers share link-state information rather than just routing tables, they can recalculate paths very quickly when a change is detected.
- Classless Support: Fully supports VLSM and CIDR.
- Multicast Updates: Uses multicast addresses (224.0.0.5 for all SPF routers and 224.0.0.6 for Designated Routers) to send updates.
- Event-Driven Updates: only sends updates when a change occurs in the topology (LSU).

- Authentication: Supports cleartext, MD5, and SHA authentication for secure routing updates between neighbors.

Limitations:

- Running the SPF algorithm requires significant CPU power and memory specially in large and flat (single-area) networks with many routers.
- Setting up multi-area OSPF and troubleshooting virtual links or summarization can be more difficult than simpler protocols like RIP.
- Strict Design Requirements: The requirement that all areas must connect to Area 0 can make network expansion rigid if the physical topology doesn't naturally support a "hub-and-spoke" area design.
- Reference Bandwidth Issues: Since the default reference bandwidth is 100 Mbps OSPF treats 100 Mbps, 1 Gbps, and 10 Gbps links all as a Cost of 1 unless the reference bandwidth is manually adjusted.

2.3 Intermediate System to Intermediate System (IS-IS)

IS-IS is a high-performance, link-state routing protocol. Originally created by the ISO for the OSI protocol suite, it was later adapted to support IP, becoming the backbone of choice for many of the world's largest Service Providers. It is prized for its extreme scalability, protocol-agnostic nature, and high stability under heavy load.

Type:

It uses Dijkstra's SPF (Shortest Path First) algorithm to calculate the best path. Each router maintains a Link State Database (LSDB) to ensure a synchronized view of the network. Unlike OSPF, it operates natively at Layer 2, sending updates directly over the Data Link layer without requiring an IP header.

Scope:

IS-IS is an Interior Gateway Protocol (IGP) used for routing within a single Autonomous System. While it can scale to massive sizes, it is predominantly found in Service Provider cores and large-scale Data Centers. It uses a flexible level-based hierarchy (Level 1 for intra-area and Level 2 for inter-area backbone) rather than a rigid area-centric design.

Metric:

IS-IS uses a dimensionless Interface Cost metric.

Default Metric: Almost all implementations default to a cost of 10 per interface, regardless of bandwidth. Wide Metrics: Modern networks must use "Wide Metrics" (24-bit/32-bit values) to support high-speed links and advanced traffic engineering.

Key Features:

- Manages traffic by dividing the network into logical Areas. Unlike other protocols where individual interfaces are assigned to an area, IS-IS assigns the entire router to a specific area.
- Protocol Agnostic: Because it runs at Layer 2, IS-IS can carry routing information for IPv4, IPv6, and non-IP protocols simultaneously using the same adjacency.
- Partial Route Calculation (PRC): It can update specific prefix changes (leaf nodes) without performing a full, CPU-heavy recalculation of the entire network map.
- TLV Extensibility: It uses Type-Length-Value data fields, allowing engineers to add

new features without needing to overhaul the core protocol

- **High Stability:** Its independence from the IP stack makes it resistant to IP-based spoofing or DoS attacks and ensures it stays up even if IP configurations fail.

Limitations:

- **Operational Complexity:** Requires the configuration of NET addresses, which follow a non-IP ISO format that is often unfamiliar to enterprise engineers.
- **Troubleshooting Hurdles:** Standard IP-based tools (like ping/traceroute) cannot be used to verify the protocol's own adjacency; specialized Layer 2 commands are required.
- **Suboptimal Defaults:** The default narrow metrics and fixed cost of 10 can lead to poor routing decisions on high-speed links if not manually tuned to wide metrics.
- **Learning Curve:** It is generally considered more difficult to master than OSPF, leading to a smaller pool of qualified engineers for support.

2.4 Border Gateway Protocol (BGP)

BGP is the primary routing protocol of the global internet, often described as the “postal service” or the “glue” that holds the digital universe together. It manages how data packets are delivered between massive networks known as Autonomous Systems (ASs).

Type:

BGP is classified as a path-vector protocol. It operates over reliable TCP sessions (typically on port 179).

Scope:

Its primary role is as an Exterior Gateway Protocol (EGP) for inter-AS routing, though it is also used internally (iBGP) to ensure all routers within an AS share consistent information.

Metric:

BGP does not use traditional metrics like bandwidth or delay; instead, it uses a complex hierarchy of Path Attributes (PAs) and rule-sets to select routes. The AS_PATH is a primary attribute used for path selection and loop prevention.

Features:

- It is the only protocol capable of managing the massive global internet routing table, which now exceeds hundreds of thousands of prefixes.
- It offers granular policy-based routing control and supports multi-homing for high availability and redundancy.

Limitations:

- BGP has very slow convergence times because routers explore many invalid paths before stabilizing after a change.
- It lacks intrinsic security, making it highly vulnerable to prefix hijacking, spoofing, and TCP reset attacks.
- Routing decisions are often based on coarse attributes like the number of AS hops rather than actual link performance metrics.

3. Proposed Protocol Design - Adaptive Open Shortest Path First (AOSPF)

AOSPF extends the existing OSPF simulation with three major enhancements: adaptive composite metrics, threshold-based triggering, and enhanced LS-UPDATE packets.

3.1 Adaptive Composite Routing Metric

Define the composite cost for a path as:

$$\text{Cost} = w_1 \sum_{i \in \text{path}} \left(\frac{L_i}{L_{\max}} \right) + w_2 \sum_{i \in \text{path}} \left(-\log \left(\frac{BW_i}{BW_{\max}} \right) \right)$$

where:

- L_i = current measured delay of link i (ms)
- BW_i = bandwidth of link i (Mbps)
- $L_{\max}$ = normalization constant (max expected delay, e.g., 100 ms)
- $BW_{\max}$ = normalization constant (max bandwidth, e.g., 10000 Mbps)
- w_1, w_2 = configurable weights with $w_1 + w_2 = 1$

Comparison with OSPF Cost:

Aspect	Traditional OSPF	AOSPF
Metric Formula	[REF_BW/BW]	$w_1(L/L_{\max}) + w_2(-\log(BW/BW_{\max}))$
Delay Awareness	None	Weighted component
Bandwidth Handling	Linear inverse	Logarithmic (better differentiation)
Adaptivity	Static (manual reconfiguration)	Automatic threshold-based updates
Flooding Triggers	Topology changes only	Topology changes + metric variations > threshold
Multi-criteria	Single metric	Weighted dual-metric

3.2 Threshold-Based Triggering Mechanism

The core innovation of AOSPF is that LS-UPDATE packets are flooded when link delay variations exceed a predetermined threshold, not just on topology changes.

Threshold Logic:

```
DELAY_THRESHOLD = 0.20 # 20% change triggers update
MIN_UPDATE_INTERVAL = 500 # ms, prevents flooding storms
```

```
# Track last advertised metrics per link
last_advertised_delay = {edge: original_delay for edge in edges}
last_update_time = {edge: 0 for edge in edges}
```



```
def check_threshold(edge, current_delay, current_time):
    if current_time - last_update_time[edge] >= MIN_UPDATE_INTERVAL:
        relative_change = abs(current_delay - last_advertised_delay[edge])
                        / last_advertised_delay[edge]
        if relative_change >= DELAY_THRESHOLD:
            return True # Trigger LS-UPDATE
    return False
```

Hysteresis Prevention: To prevent flapping when delay oscillates near the threshold, a hysteresis factor (e.g., 0.8) requires the delay to cross below $0.8 \times$ threshold before re-triggering.

3.3 Enhanced Control Packet Formats

3.3.1 Common Packet Header (Unchanged from OSPF)

Field	Size	Description
Version	8 bits	Protocol version number
Packet Type	8 bits	0x01=HELLO, 0x02=LS-UPDATE, etc.
Flags	8 bits	Includes THRESHOLD_CROSSED flag
Hop Limit	8 bits	Flood-control field
Sequence Number	32 bits	Replay prevention
Sender ID	32 bits	Unique router identifier
Payload Length	16 bits	TLV payload length

3.3.2 Enhanced Link-Metric TLV (AOSPF Extension)

Field	Size	Description
Neighbor ID	32 bits	Adjacent router identifier
Delay (current)	16 bits	Current link delay (μ s)
Delay (baseline)	16 bits	Baseline for threshold calculation
Bandwidth	32 bits	Link bandwidth (Mbps)
Last Update Timestamp	32 bits	Time of last advertisement (ms)
Flags	8 bits	THRESHOLD_CROSSED, STALE flags
Reserved	8 bits	Future use

3.3.3 THRESHOLD_CROSSED Flag

The THRESHOLD_CROSSED flag in the packet header serves critical functions:

1. **Distinguishes Update Types:** Receiving routers know whether this is a metric adjustment (keep adjacencies) or topology change (may need reset)
2. **Enables Different Rate Limits:** Metric updates have 500ms minimum interval; topology updates have 100ms
3. **Supports Hysteresis Logic:** Tracks whether link is in hysteresis zone
4. **Prevents Unnecessary Adjacency Resets:** Metric adjustments don't trigger

neighbor state machine resets

5. **Debugging and Analytics:** Separates metric-triggered vs topology-triggered floods in logs

3.4 Modified Simulation Components

The original OSPF simulation had the following structure, with modifications shown for AOSPF:

3.4.1 Original OSPF Cost Calculation (in `_load_topology`)

```
REFERENCE_BW_MBPS = 1000
cost = max(1, math.ceil(REFERENCE_BW_MBPS / bw_mbps))
self.G.add_edge(u, v, cost=cost, bandwidth=bw_str, delay=delay)
```

3.4.2 AOSPF Modified Cost Calculation

```
# AOSPF Parameters
L_MAX = 100.0      # Maximum expected delay (ms)
BW_MAX = 10000.0   # Maximum bandwidth (Mbps)
W1 = 0.4           # Latency weight
W2 = 0.6           # Bandwidth weight

def compute_aospf_cost(delay_ms, bandwidth_mbps):
    """Adaptive OSPF composite cost calculation"""
    latency_term = delay_ms / L_MAX
    bandwidth_term = -math.log(bandwidth_mbps / BW_MAX) if bandwidth_mbps
    > 0 else 10
    cost = W1 * latency_term + W2 * bandwidth_term
    return max(1, math.ceil(cost))
```

3.4.3 Original `get_current_cost` Function

```
def get_current_cost(u_node, v_node, eval_time):
    e_tuple = tuple(sorted((u_node, v_node)))
    base_cost = self.original_edges_data[e_tuple]['cost']
    for mod_edge, mod_cost, timestamp in self.cost_changes:
        if mod_edge == e_tuple and timestamp <= eval_time:
            base_cost = mod_cost
    return base_cost
```

3.4.4 AOSPF Enhanced `get_current_cost` with Delay Awareness

```
def get_current_cost(u_node, v_node, eval_time):
    e_tuple = tuple(sorted((u_node, v_node)))
    # Get current delay (may have changed due to threshold triggers)
    current_delay = get_current_delay(u_node, v_node, eval_time)
    base_bandwidth = self.original_edges_data[e_tuple]['bandwidth_mbps']

    # Compute composite cost with current delay
    latency_term = current_delay / L_MAX
    bandwidth_term = -math.log(base_bandwidth / BW_MAX)
    cost = W1 * latency_term + W2 * bandwidth_term
```

```
# Override with manual cost changes if any
for mod_edge, mod_cost, timestamp in self.cost_changes:
    if mod_edge == e_tuple and timestamp <= eval_time:
        cost = mod_cost
return max(1, math.ceil(cost))
```

3.4.5 New Threshold Monitoring in Simulation Loop

```
# Track last advertised metrics per link
last_advertised_delay = {edge: original_delay for edge in edges}
last_update_time = {edge: 0 for edge in edges}

# During simulation, when delay changes:
current_delay = get_current_delay(u, v, current_time)
last_delay = last_advertised_delay[edge_tuple]

if current_time - last_update_time[edge_tuple] >= MIN_UPDATE_INTERVAL:
    relative_change = abs(current_delay - last_delay) / last_delay
    if relative_change >= DELAY_THRESHOLD:
        # Trigger LS-UPDATE flood with THRESHOLD_CROSSED flag
        trigger_adaptive_ls_update(u, v, current_delay, baseline=last_delay)
        last_advertised_delay[edge_tuple] = current_delay
        last_update_time[edge_tuple] = current_time
        self.logs_database.append({
            "time": current_time,
            "text": f"Threshold crossed: delay changed {relative_change*100:.1f}% on {edge_tuple}",
            "type": "threshold_crossed"
        })
```

3.5 LS-UPDATE Flooding on Threshold Crossings

When a threshold is crossed, the incident routers generate LS-UPDATE packets with:

1. The THRESHOLD_CROSSED flag set in the packet header
2. Both current and baseline delay values in the Link-Metric TLV
3. The current timestamp for rate limiting
4. Incremented sequence number to ensure propagation

Receiving routers process these updates by:

1. Updating their LSDB with the new delay values
2. NOT resetting adjacency timers (since this is a metric change, not topology change)
3. Recomputing routes via Dijkstra with the new composite costs
4. Propagating the update to other neighbors (standard flooding)

4. Route Computation with Adaptive Metrics

Each router maintains a Link-State Database (LSDB) containing topology information. When an LS-UPDATE is received (either from topology change or threshold crossing), the

router recomputes routes using Dijkstra's algorithm with the composite cost function.

4.1 Modified SPF Computation

```
def compute_routes(self):
    """Compute routing table using composite AOSPF costs"""
    G = nx.Graph()
    for router, links in self.lsd.items():
        for neighbor, metrics in links.items():
            # Get current composite cost
            current_delay = metrics.get('current_delay')
            bandwidth = metrics.get('bandwidth')
            cost = W1 * (current_delay / L_MAX) + \
                W2 * (-math.log(bandwidth / BW_MAX))
            G.add_edge(router, neighbor, weight=max(1, math.ceil(cost)))

    # Run Dijkstra from this router
    distances, paths = nx.single_source_dijkstra(G, self.id)
    self.update_routing_table(paths, distances)
```

5. Failure Detection and Recovery

AOSPF inherits OSPF's failure detection mechanisms and enhances them:

- **HELLO/Keepalive Timers:** Periodic HELLO packets every `hello_interval` (default: 3000ms). Missed HELLO within `dead_interval` ($4 \times \text{hello}$) triggers link-down event.
- **Threshold-Based Reactivity:** Even without failures, significant delay variations (greater than 20%) trigger network-wide updates, allowing routes to adapt to congestion before failure occurs.
- **Triggered LS Updates:** On link-down OR threshold crossing, routers immediately flood LS-UPDATE with `THRESHOLD_CROSSED` flag.
- **Hold-down/Wait:** Brief hold-down period (configurable, default 1000ms) before reinstating flapping links.

6. Security Enhancements

AOSPF incorporates a comprehensive security layer based on Hash-based Message Authentication Code (HMAC) to protect against routing attacks. This section details the HMAC mechanism and explains how it improves protocol robustness.

6.1 Overview of HMAC

HMAC (Hash-based Message Authentication Code) is a keyed cryptographic hash function that provides both message authentication and integrity verification. In AOSPF, all control packets (HELLO, LS-UPDATE, LS-ACK) are signed using HMAC-SHA256 with a pre-shared secret key known only to legitimate routers. The HMAC computation is defined as:

$$\text{HMAC}(K, m) = \text{SHA256}\left((K \oplus \text{opad}) \parallel \text{SHA256}((K \oplus \text{ipad}) \parallel m)\right)$$

where:

- K is the shared secret key
- m is the canonical representation of the packet header and payload
- ipad and opad are fixed padding constants
- $\parallel$ denotes concatenation

6.2 Canonical Representation for AOSPF Packets

To ensure deterministic HMAC computation, AOSPF constructs a canonical string from the packet's immutable fields. For an LS-UPDATE packet, the canonical representation includes:

```
canonical = f"{packet.version}|{packet.type}|{packet.flags}|"
           f"{packet.sequence_num}|{packet.sender_id}|"
           f"{sorted(packet.link_metrics.items())}"
```

The sorted order of link metrics guarantees that two semantically identical packets produce the same HMAC tag regardless of the order of TLVs.

6.3 HMAC Signing and Verification Procedures

The following pseudocode describes how AOSPF routers sign outgoing packets and verify incoming packets.

Algorithm 1: SignPacket - generate HMAC tag before transmission

Input: Packet P (header + payload), shared key K

Output: Signed packet P_{signed} with HMAC tag

```
canonical <- BuildCanonical(P)
tag <- HMAC-SHA256(K, canonical)
P.hmac <- tag
return P
```

Algorithm 2: VerifyPacket - authenticate and check integrity

Input: Received packet P_{rcv} , shared key K

Output: Boolean (valid/invalid) and optional reason

```
if hmac not in P_rcv:
    return (False, "No HMAC tag")

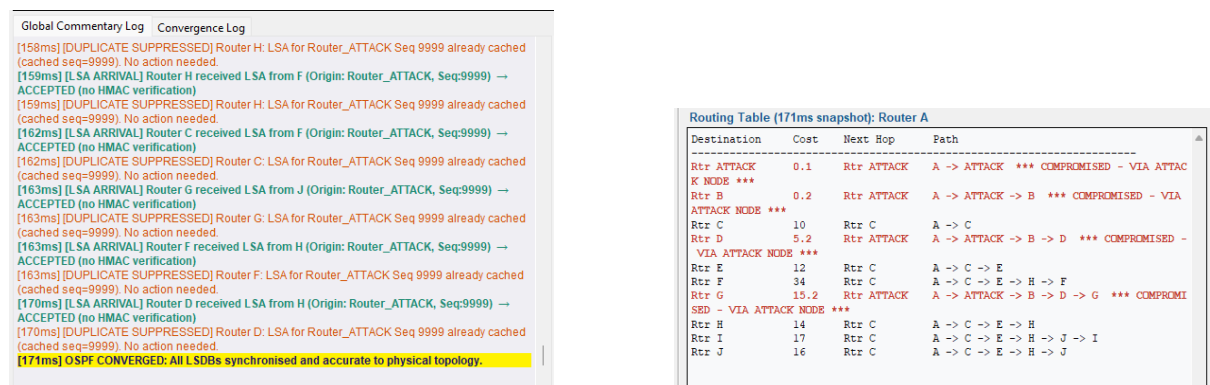
canonical <- BuildCanonical(P_rcv)
expected <- HMAC-SHA256(K, canonical)

if compare_digest(expected, P_rcv.hmac):
    return (True, "Valid")
else:
    return (False, "HMAC mismatch - forgery detected")
```

6.4 How HMAC Improves Protocol Robustness

Compared to an unprotected OSPF implementation (as in the baseline simulation), the integration of HMAC provides the following security guarantees:

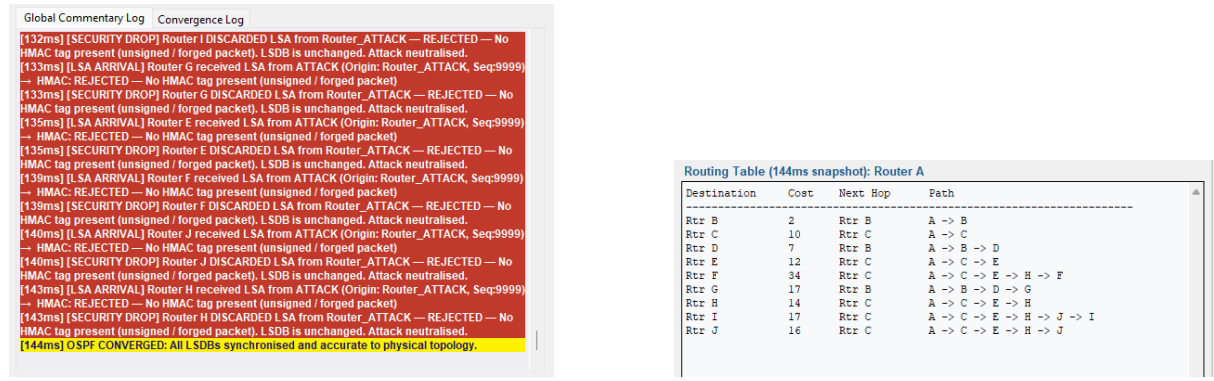
1. **Authentication of Origin:** A router can cryptographically verify that an LS-UPDATE originated from a legitimate router that possesses the shared secret. Without HMAC, any attacker on a broadcast link can inject forged LSAs claiming to be any router.
2. **Integrity Protection:** The HMAC covers the entire packet header and payload. Any modification of fields (e.g., changing sequence numbers, metric values, or neighbor lists) will cause verification to fail. This prevents man-in-the-middle attacks that alter routing information in transit.
3. **Replay Attack Prevention:** The packet header includes a monotonically increasing sequence number. Combined with HMAC, receivers can detect and discard old or replayed packets because the sequence number is covered by the HMAC and cannot be forged.
4. **Denial-of-Service Mitigation:** AOSPF implements rate limiting on LS-UPDATE packets (MIN_UPDATE_INTERVAL = 500 ms). Even if an attacker manages to forge a packet with a valid HMAC (which requires key compromise), the rate limit prevents flooding storms. Moreover, invalid packets are dropped early in processing, reducing CPU load from cryptographic operations.
5. **Protection Against Metric Manipulation:** In the insecure OSPF simulation, an attacker can inject a fake LSA with artificially low costs (e.g., 0.1) to hijack routing. With HMAC, such a packet would be rejected because the attacker cannot generate the correct HMAC tag. This directly addresses the vulnerability demonstrated in the AOSPF adaptation to congestion.
6. **Forensic Traceability:** Every rejected packet is logged with the reason (no HMAC, mismatch, stale sequence number). This provides network operators with clear evidence of attempted attacks, enabling proactive responses.



(a) Fake LSA getting accepted by Routers

(b) Routing table compromised

Figure 1: Without HMAC verification



(a) Fake LSA rejected by HMAC verification

(b) Correct routing table

Figure 2: With HMAC verification

6.5 Security Impact on Adaptive Metrics

The adaptive metric updates (threshold-triggered LS-UPDATES) are also protected by HMAC. This ensures that even though routers autonomously react to delay variations, an attacker cannot force a routing change by spoofing a threshold-crossing event. The combination of HMAC and rate limiting makes AOSPF resilient to both external forgery and insider attacks where a compromised router might try to destabilise the network by generating excessive metric updates.

7. Simulation Implementation

The complete simulation implementation, including all source code, configuration files, and experiment scripts, is available at:

<https://github.com/Achintha-Bandara/Routing-Protocol-Design.git>

The AOSPF simulation extends the provided OSPF codebase:

7.1 Modified Data Structures

```
self.last_advertised_delay = {}      # Per-link last advertised delay
self.last_update_time = {}          # Per-link last update timestamp
self.threshold_config = {           # Threshold parameters
    'delay_threshold': 0.20,         # 20% change triggers update
    'min_update_interval': 500,      # ms between updates
    'hysteresis_factor': 0.8         # Exit threshold = 0.8 * entry
}
self.baseline_delays = {}           # Initial calibration values
```

7.2 Modified Event Types

New event type added to the discrete event simulator:

- **DELAY_MEASURE:** Periodic delay sampling on each link
- **THRESHOLD_CROSSED:** Triggers LS-UPDATE flood (new)

7.3 Modified Methods

1. `_load_topology()`: Changed to compute AOSPF composite costs instead of OSPF static costs
2. `get_current_cost()`: Enhanced to consider current delay values
3. `run_continuous_event_simulation()`: Added threshold monitoring loop
4. `process_packet()`: Extended to handle THRESHOLD_CROSSED flag

7.4 New Methods Added

1. `check_threshold_and_flood()`: Evaluates delay changes and triggers updates
2. `trigger_adaptive_ls_update()`: Generates LS-UPDATE with threshold-crossed semantics
3. `process_adaptive_ls_update()`: Handles metric-triggered updates differently from topology updates

8. Experiment Matrix and Parameters

8.1 Key Parameters

1. Network Size (N): 10, 20, 50 nodes
2. Topology Density: average degree 3 (sparse), 5 (moderate), 8 (dense)
3. Delay Threshold (T): 10%, 20%, 30%
4. Metric Weights: $(w_1, w_2) = (0.3, 0.7), (0.5, 0.5), (0.7, 0.3)$
5. Link Failure Rate: 0%, 5%, 10%
6. Hello Interval: 1000ms, 3000ms, 5000ms
7. Protocol Variant: OSPF (baseline) vs AOSPF

8.2 Performance Metrics

- **Convergence Time**: Time from topology change or threshold crossing until all routers have consistent LSDB
- **Control Overhead**: Number of LS-UPDATE packets (separated by type: topology vs threshold)
- **Route Optimality**: Average composite cost of routes vs shortest possible
- **Adaptation Latency**: Time from delay change exceeding threshold to route update completion
- **Update Frequency**: Number of threshold-triggered updates per link per second

9. Results

9.1 Comparison: AOSPF vs OSPF

Metric	OSPF	AOSPF
Convergence after failure	Fast (link-state flooding)	Fast (link-state flooding)
Adaptation to congestion	None (static costs)	Fast (threshold-triggered updates)
Path quality with congestion	Poor (uses static costs)	Good (adapts to current delay)
Control overhead (steady)	Low (event-driven)	Low + threshold updates
Control overhead (congestion)	Low (no updates)	Moderate (updates on threshold crossings)
High-speed link differentiation	Poor (all $\geq 100\text{Mbps}$ cost=1)	Good (logarithmic scaling)

9.2 Specific Expectations

1. **Faster Adaptation to Congestion:** When link delay increases due to congestion, AOSPF detects the change within the measurement interval and floods LS-UPDATE when threshold is crossed. OSPF never adapts.
2. **Controlled Overhead:** With 20% threshold, links with stable delay produce no updates. Links with gradual delay changes produce updates only when cumulative change exceeds 20%, not on every fluctuation.
3. **Better Path Selection:** AOSPF can trade off between low-latency and high-bandwidth paths based on w_1 and w_2 weights. OSPF only considers bandwidth.
4. **Stability with Hysteresis:** The hysteresis factor (0.8) prevents flapping when delay oscillates near the threshold boundary.
5. **Convergence Time:** Similar to OSPF for topology changes; for metric changes, convergence equals detection time + flooding time + SPF computation.

10. Limitations and Extensions

10.1 Limitations

- Threshold configuration requires network knowledge (optimal threshold may vary by link type)
item Assumes reliable control packet delivery (no channel errors)
- Delay measurements assume synchronized clocks between routers
- Weights w_1 and w_2 are static; not adaptive to network conditions
- Security uses pre-shared keys (not PKI)

10.2 Extensions

- Adaptive weight tuning based on real-time network conditions
- Machine learning for optimal threshold prediction per link

- Multi-metric extension: add reliability, jitter, energy consumption
- Integration with SDN controllers for global threshold optimization
- Implementation in real router platforms (FRRouting, Quagga)

11. Router State Machine (Modified for AOSPF)

stateDiagram-v2

```

[*] --> Idle
Idle --> SendHELLO: HELLO timer expires
Idle --> ProcessPKT: receive packet

ProcessPKT --> UpdateDB: valid LS-UPDATE
UpdateDB --> CheckThreshold: metric change
CheckThreshold --> Recompute: change > threshold
CheckThreshold --> Idle: change <= threshold
Recompute --> Idle

Idle --> MeasureDelay: periodic measurement
MeasureDelay --> CheckThresholdLocal: new delay value
CheckThresholdLocal --> SendLSU: threshold crossed
SendLSU --> Idle

Idle --> LinkDown: missing HELLO / failure
LinkDown --> SendLSU: immediate flood (THRESHOLD_CROSSED=0)
```

New states compared to OSPF:

- **MeasureDelay**: Periodic delay sampling on each link
- **CheckThresholdLocal**: Evaluate if delay change exceeds configured threshold
- **THRESHOLD_CROSSED flag**: Distinguishes metric-triggered from topology-triggered floods

12. Conclusion

This report presented Adaptive Open Shortest Path First (AOSPF), an extension to traditional OSPF that introduces:

1. A composite cost function combining delay and bandwidth metrics
2. Threshold-based triggering where link delay variations exceeding a predetermined threshold cause LS-UPDATE floods
3. Enhanced packet formats with THRESHOLD_CROSSED flag for update type differentiation
4. Rate limiting and hysteresis to prevent update storms

The implementation modifies an existing OSPF discrete-event simulation written in Python with NetworkX and tkinter. Key code changes include:

- Modified `_load_topology()` for composite cost calculation
- Enhanced `get_current_cost()` to consider real-time delay
- Added threshold monitoring in `run_continuous_event_simulation()`
- New methods for threshold detection and adaptive LS-UPDATE flooding

AOSPF addresses OSPF's fundamental limitation: the inability to adapt to changing network conditions. By flooding updates when delay variations exceed a threshold, AOSPF enables dynamic route optimization while controlling control-plane overhead through rate limiting and hysteresis.

References

- [1] J. Moy, "OSPF Version 2," RFC 2328, Apr. 1998. <https://datatracker.ietf.org/doc/html/rfc2328>
- [2] R. Coltun, D. Ferguson, J. Moy, and A. Lindem, "OSPF for IPv6," RFC 5340, Jul. 2008. <https://datatracker.ietf.org/doc/html/rfc5340>
- [3] G. Malkin, "RIP Version 2," RFC 2453, Nov. 1998. <https://datatracker.ietf.org/doc/html/rfc2453>
- [4] S. Murphy, "BGP Security Vulnerabilities Analysis," RFC 4272, Jan. 2006. <https://datatracker.ietf.org/doc/html/rfc4272>
- [5] Y. Rekhter, T. Li, and S. Hares, "A Border Gateway Protocol 4 (BGP-4)," RFC 4271, Jan. 2006. <https://datatracker.ietf.org/doc/html/rfc4271>
- [6] C. Perkins, E. Belding-Royer, and S. Das, "Ad hoc On-Demand Distance Vector (AODV) Routing," RFC 3561, Jul. 2003. <https://datatracker.ietf.org/doc/html/rfc3561>
- [7] S. Kent and K. Seo, "Security Architecture for the Internet Protocol," RFC 4301, Dec. 2005. <https://datatracker.ietf.org/doc/html/rfc4301>